JAVA OOP LABORATORY EXERCISE
CAMPUS CAFÉ ORDERING SYSTEM

ITS 202: Object-Oriented Programming

Submitted by:

Renato Anthony M. Soligan
Jusuel M. Gabas

Section: C-3:30-6:00 PM

NOVEMBER 2026

1. Introduction

The Campus Café Ordering System is a Java console-based program that manages sample customer orders. It supports three types of menu items: beverages, pastries, and meals. Each item has its own pricing rules. The system calculates order subtotals, applies a 10% student discount, deducts a voucher, displays receipts, and awards loyalty points based on the final payment.

The program is organized into seven Java classes: MenuItem, Beverage, Pastry, Meal, Customer, Order, and CafeApp. These classes work together to represent menu items, customers, and orders. This project focuses on Java programming and Object-Oriented Programming concepts; it does not include a database.

2. OOP Concepts Used

Classes and Objects

The program uses seven classes: MenuItem, Beverage, Pastry, Meal, Customer, Order, and CafeApp. CafeApp creates customer objects and menu-item objects, then uses Order objects to process and display the sample transactions.

Abstraction

MenuItem is an abstract class. It contains shared menu-item information such as itemName, price, and code. It also declares the abstract computePrice() method, requiring each subclass to provide its own price calculation.

Encapsulation

The classes organize their data using private and protected fields. For example, Customer keeps customerName, isStudent, and points private, while MenuItem keeps code private. Getter and update methods provide controlled access to these values. Price and side-order validation also prevent certain invalid negative values.

Inheritance

Beverage, Pastry, and Meal extend MenuItem. They inherit common item information and reuse the parent class constructor. Each subclass adds its own attributes, such as selectedSize for Beverage, heated for Pastry, and largeSize and sideOrders for Meal.

Polymorphism and Method Overriding

The Order class stores menu items using MenuItem references. When it calls computePrice() or getDescription(), Java executes the overridden method belonging to the actual object type, such as Beverage, Pastry, or Meal. This allows different menu-item types to be processed through the same parent-class reference.

Method Overloading

Customer overloads earnPoints() with int and double parameters. The int version adds a specified number of points, while the double version calculates points from a payment amount. Order also overloads calculateTotal() to support a voucher amount, and displayReceipt() to display a receipt with or without a voucher.

Conditional Statements

The program uses if and if-else statements to validate prices and quantities, select beverage sizes, add heating or upsizing charges, check available order slots, apply student discounts, and validate the final amount after a voucher deduction.

Constructors

MenuItem, Beverage, Pastry, and Meal provide default and parameterized constructors to initialize menu items. Customer provides three constructors for different customer details. Order uses a parameterized constructor to set the order reference number and customer.

3. Program Demonstration

The first sample order is for Ana Reyes, who is marked as a student. It contains an iced latte in large size, a heated ensaymada, and an upsized chicken adobo with one side order. The receipt displays the items, subtotal, 10% student discount, total payment, and the customer's updated loyalty points.

The second sample order is for Ben Cruz, a regular customer. It contains a medium hot chocolate and a regular beef tapa. The CafeApp code first attempts to assign a negative price to the hot chocolate, which triggers the price-validation message, and then assigns a valid price. The second receipt applies a 20-unit voucher before calculating loyalty points.

The CafeApp code also creates a cookie object, but it does not add that object to either order. The current version does not attempt to add a fourth item, so it does not demonstrate the order-limit message.

4. Challenges and Solutions

One challenge in developing the system is allowing the Order class to handle different menu-item types without writing a separate order process for each type. This is addressed by using MenuItem as an abstract parent class and storing the selected items as MenuItem references. Method overriding allows each item to calculate its own price and provide its own description.

Another challenge is preventing invalid values from affecting the order. The program checks prices and side-order quantities before accepting them. It also checks whether the order slots are already occupied and prevents a voucher deduction from producing a negative final amount.

5. Program Output Screenshot

Insert a screenshot of the actual console output after compiling and running CafeApp. The screenshot should show the first and second receipts, the negative-price validation message, the voucher deduction, and the loyalty-point tota

